Robotics Lab: Homework 4

Control a mobile robot to follow a trajectory

Gaetano Torella
Stefano Riccardi
Marco Maffeo
Antonio D'Angelo

GitHub: https://github.com/marcomaffeo/homework4.git

This document contains the homework 4 of the Robotics Lab class.

Control a mobile robot to follow a trajectory

The goal of this homework is to implement an autonomous navigation software framework to control a mobile robot. The rl_fra2mo_description and fra2mo_2dnav packages must be used as a starting point for the simulation. The student is requested to address the following points and provide a detailed report of the methods employed. In addition, a personal GitHub repo with all the developed code must be shared with the instructor. The report is due in one week from the homework release.

1. **Construct a gazebo world and spawn the mobile robot in a given pose**

a) Launch the Gazebo simulation and spawn the mobile robot in the world rl_racefield in the pose

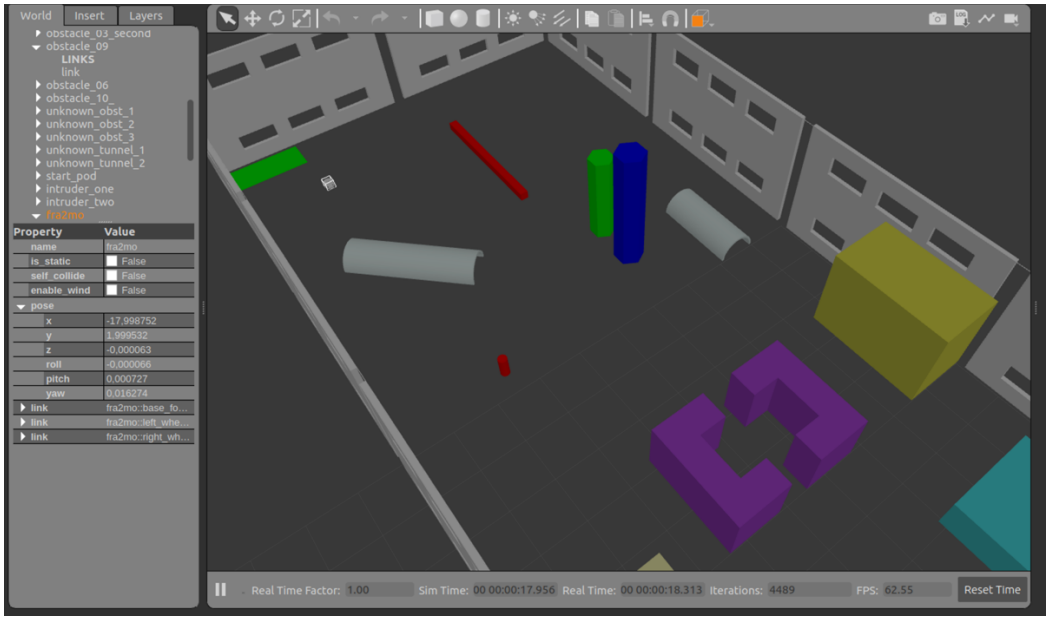
$$x = -3 \quad y = 5 \quad yaw = -90\,\mathrm{deg}$$

with respect to the map frame. The argument for the yaw in the call of spawn_model is Y.

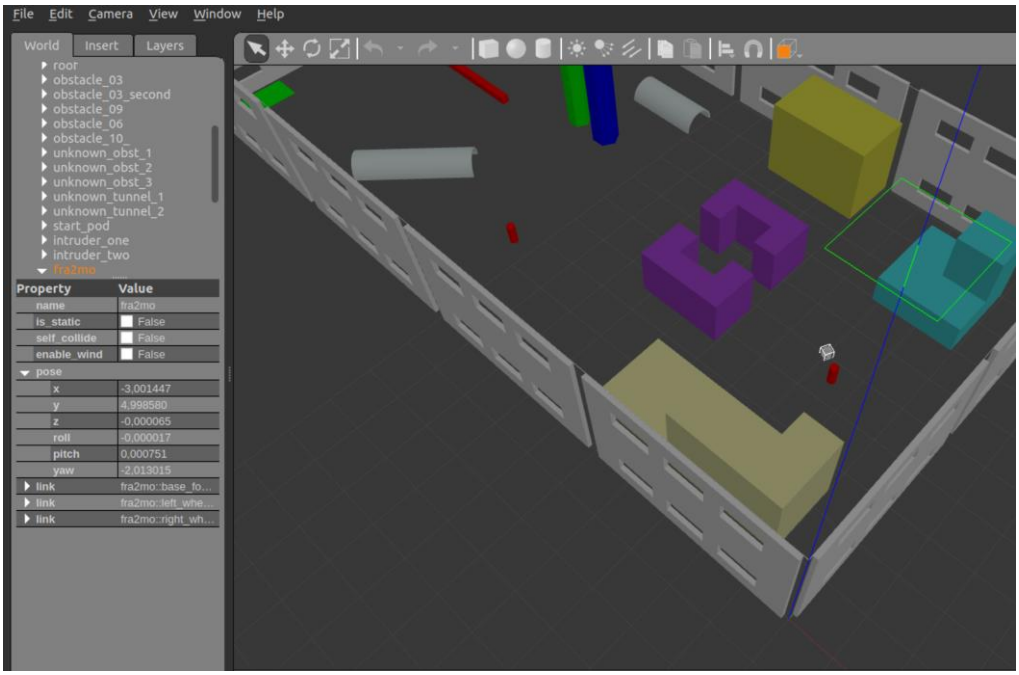
Inside the spawn_fra2mo_gazebo.launch we add the argument for the yaw parameter and we change the values of x_pos and y_pos; after that we add the argument for the yaw the node which spawn the model

```
  <arg name="x_pos" default="-3.0"/>
  <arg name="y_pos" default="5.0"/>
  <arg name="z_pos" default="0.1"/>
  <arg name="yaw" default="-1.56"/>
. . .
  args="-urdf -model fra2mo -x $(arg x_pos) -y $(arg y_pos) -z $(arg z_pos) -Y $(arg yaw) -param
robot_description"/>
```

Before:



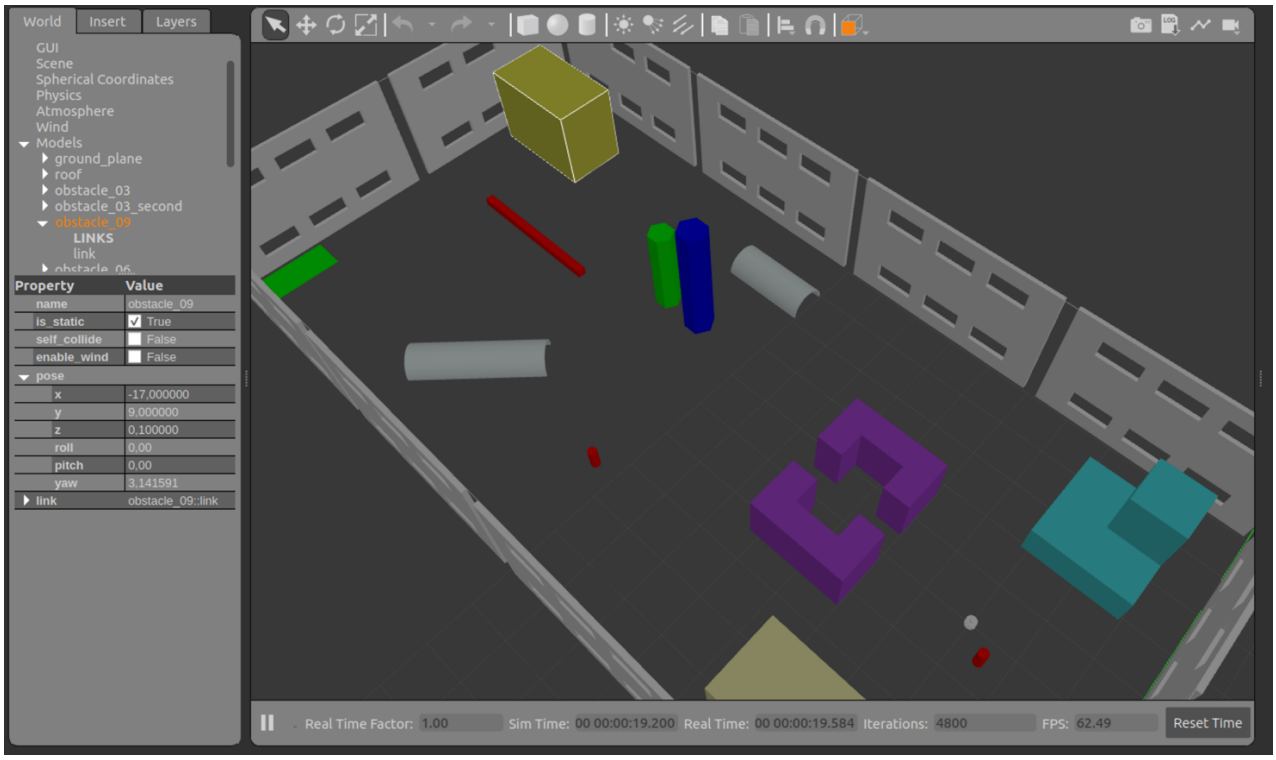
After:



b) Modify the world file of rl_racefield moving the obstacle 9 in position:

$$x = -17 \quad y = 9 \quad z = 0.1 \quad yaw = 3.14$$

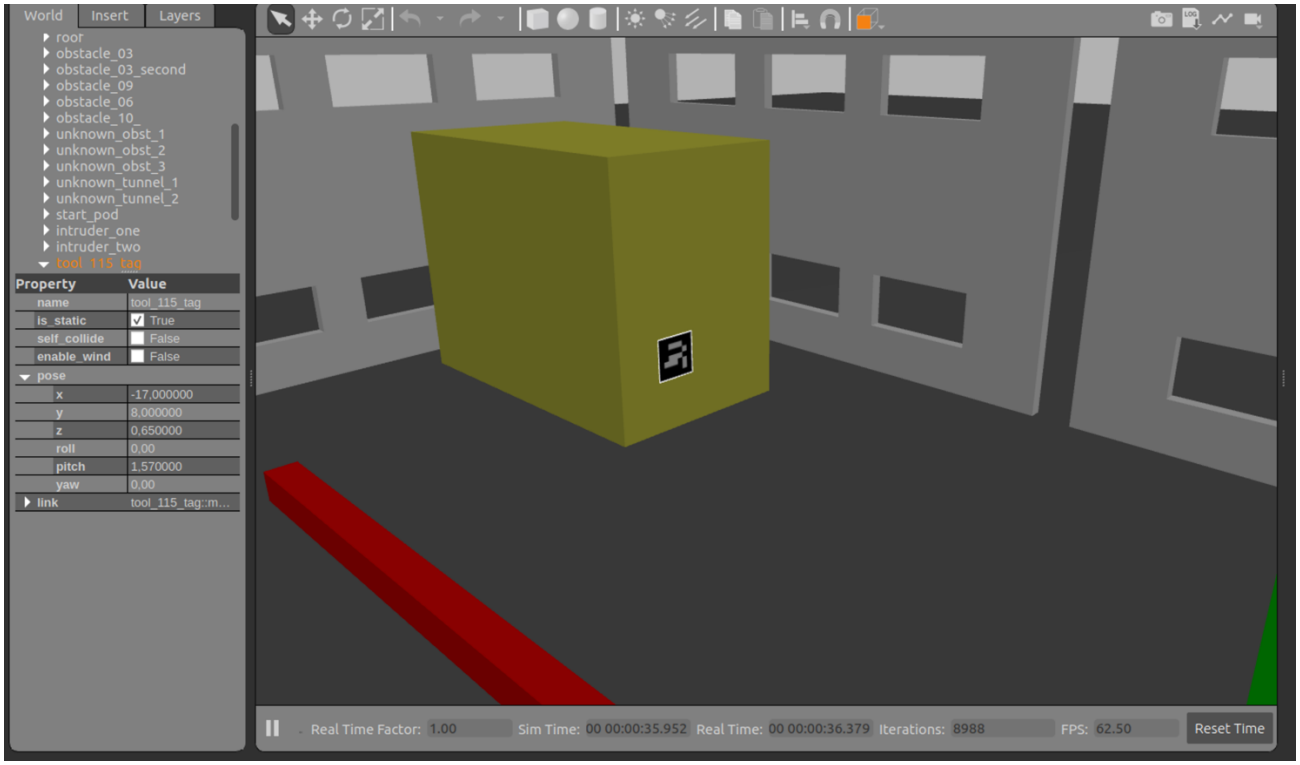
```
<include>
  <name>obstacle_09</name>
  <pose> -17 9 0.1 0 0 3.14159</pose>
  <uri>model://obstacle_09</uri>
</include>
```



c) Place the ArUco marker number 115 on obstacle 9 in an appropriate position, such that it is visible by the mobile robot's camera when it comes in the proximity of the object.

First of all we generate the ArUco marker 115 on the website https://chev.me/arucogen/ in the Original ArUco dictionary, we download the .svg file and we converted it into a .png file with the dimension of 1200x1200 pixel; after that we create a new model called marker_115 inside the models folder of rl_racefield and we uncomment properly the AR marker lines inside the file rl_race_field.world changing the values of the pose:

```
<!-- AR markers -->
<include>
  <name>tool 115 tag</name>
  <uri>model://marker 115</uri>
  <pose>-17 8 0.4 0 1.57 0</pose>
</include>
```



2. Place static tf acting as goals and get their pose to enable an autonomous navigation task

a) Insert 4 static tf acting as goals in the following poses with respect to the map frame:

- Goal_1: $x = -10 \quad y = 3 \quad yaw = 0\,\text{deg}$
- Goal_2: $x = -15 \quad y = 7 \quad yaw = 30\,\text{deg}$
- Goal_3: $x = -6 \quad y = 8 \quad yaw = 180\,\text{deg}$
- Goal_4: $x = -17.5 \quad y = 3 \quad yaw = 75\,\text{deg}$

Follow the example provided in the launch file rl_fra2mo_description /launch/ spawn_fra2mo_gazebo.launch of the simulation.

First of all we compute the quaternion values of the 4 goals starting from the Euler angles; after that we add four tf in the spawn_fra2mo_gazebo.launch file:

```
<node pkg="tf" type="static_transform_publisher" name="goal_1_pub" args="-10 3 0 0 0 0 1 map goal1 100"/>
<node pkg="tf" type="static_transform_publisher" name="goal_2_pub" args="-15 7 0 0 0 0.259 0.966 map goal2
100"/>
<node pkg="tf" type="static_transform_publisher" name="goal_3_pub" args="-6 8 0 0 0 1 0 map goal3 100"/>
<node pkg="tf" type="static_transform_publisher" name="goal_4_pub" args="-17.5 3 0 0 0 0.609 0.793 map
goal4 100"/>
```

b) Following the example code in fra2mo_2dnav/src/tf_nav.cpp, implement tf listeners to get target poses and print them to the terminal as debug.

In the tf_nav files (.ccp and .h) we add four void function called goal_listener, one for each goal starting from the given one.

```
void TF_NAV::goal2_listener() {
    ros::Rate r( 1 );
    tf::TransformListener listener;
    tf::StampedTransform transform;

    while ( ros::ok() )
    {
        try
        {
            listener.waitForTransform( "map", "goal2", ros::Time( 0 ), ros::Duration( 10.0 ) );
            listener.lookupTransform( "map", "goal2", ros::Time( 0 ), transform );
        }
        catch( tf::TransformException &ex )
        {
            ROS_ERROR("%s", ex.what());
            r.sleep();
            continue;
        }

         _goal_pos2 << transform.getOrigin().x(), transform.getOrigin().y(),
transform.getOrigin().z();
         _goal_or2 << transform.getRotation().w(),  transform.getRotation().x(),
transform.getRotation().y(), transform.getRotation().z();
        std::cout<< "goal2: "<<  _goal_pos << endl;
        r.sleep();
    }
}
```

c) Using move_base, send goals to the mobile platform in a given order. Go to the next one once the robot has arrived at the current goal. The order of the explored goals must be Goal 3 → Goal 4 → Goal 2 → Goal 1. Use the Action Client communication protocol to get the feedback from move_base. Record a bagfile of the executed robot trajectory and plot it as a result.

Frist of all we add 3 others _goal_pos and _goal_or variables in the TF_NAV constructor to contain the values of the goal positions and orientations that came from the listener. After that we add three nested if inside the send_goal function in order to send the new goal, once the previous pose is reached, following the order of the explored goals. As follow we can see the code just for two goals.

```
        if ( cmd == 1) {
            MoveBaseClient ac("move base", true);
            while(!ac.waitForServer(ros::Duration(5.0))){
            ROS_INFO("Waiting for the move_base action server to come up");
            }
            goal.target_pose.header.frame_id = "map";
            goal.target_pose.header.stamp = ros::Time::now();

            goal.target_pose.pose.position.x = _goal_pos3[0];
            goal.target_pose.pose.position.y = _goal_pos3[1];
            goal.target_pose.pose.position.z = _goal_pos3[2];

            goal.target_pose.pose.orientation.w = _goal_or3[0];
            goal.target_pose.pose.orientation.x = _goal_or3[1];
            goal.target_pose.pose.orientation.y = _goal_or3[2];
            goal.target_pose.pose.orientation.z = _goal_or3[3];

            ROS_INFO("Sending goal3");
            ac.sendGoal(goal);

            ac.waitForResult();

            if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED){
                ROS_INFO("The mobile robot arrived in the TF goal3");
                goal.target_pose.header.frame_id = "map";
                goal.target_pose.header.stamp = ros::Time::now();

                goal.target_pose.pose.position.x =  goal_pos4[0];
                goal.target_pose.pose.position.y =  goal_pos4[1];
                goal.target_pose.pose.position.z =  goal_pos4[2];

                goal.target_pose.pose.orientation.w =  goal_or4[0];
                goal.target_pose.pose.orientation.x =  goal_or4[1];
                goal.target_pose.pose.orientation.y =  goal_or4[2];
                goal.target_pose.pose.orientation.z =  goal_or4[3];

                ROS_INFO("Sending goal4");

                ac.sendGoal(goal);

                  ac.waitForResult();
. . .
```

We record a bag file of the topic /fra2mo/pose, upload it on rqt_bag and show the plot of pose/position/x and y and a video that you can find on the GitHub repo

3. Map the environment tuning the navigation stack's parameters

a) Modify, add, remove, or change pose, the previous goals to get a complete map of the environment.

To get a complete map of the environment we decide to add other 3 tf goals:

Goal 5:	x = -18.6	y = 9.6	yaw = 180 deg
Goal 6:	x = -0.6	y = 0.6	yaw = 0 deg
Goal 7:	x = -0.5	y = 9.4	yaw = 0 deg

```
<node pkg="tf" type="static transform publisher" name="goal 5 pub" args="-18.6 9.6 0 0 0 1 0 map
goal5 100"/>
<node pkg="tf" type="static transform publisher" name="goal 6 pub" args="-0.6 0.6 0 0 0 0 1 map
goal6 100"/>
<node pkg="tf" type="static_transform_publisher" name="goal_7_pub" args="-0.5 9.4 0 0 0 0 1 map
goal7 100"/>
```

For this reason, we add 3 tf listners and 3 others _goal_pos and _goal_or variable inside the tf_nav files. However, as introducing further nested 'if' loops would lead to confusion within the code, we have opted to create a function that incorporates the current desired position and orientation values into the 'goal' variable.

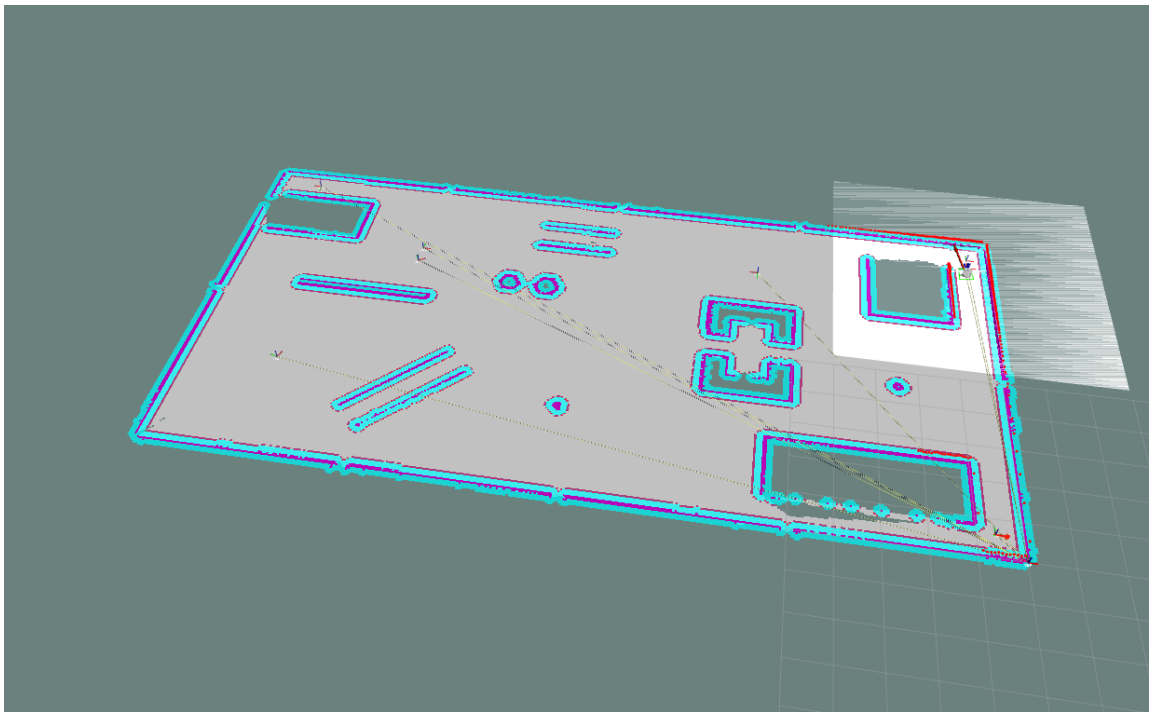
```
void TF_NAV::sender (Eigen::Vector3d _pos, Eigen::Vector4d _or, int num,
move_base_msgs::MoveBaseGoal &goal){

    goal.target_pose.header.frame_id = "map";
    goal.target_pose.header.stamp = ros::Time::now();
```

```
    goal.target_pose.pose.position.x = _pos[0];
    goal.target_pose.pose.position.y = _pos[1];
    goal.target_pose.pose.position.z = _pos[2];

    goal.target_pose.pose.orientation.w = _or[0];
    goal.target_pose.pose.orientation.x = _or[1];
    goal.target_pose.pose.orientation.y = _or[2];
    goal.target_pose.pose.orientation.z = _or[3];

    ROS_INFO("Sending goal: %d",num);
}
```

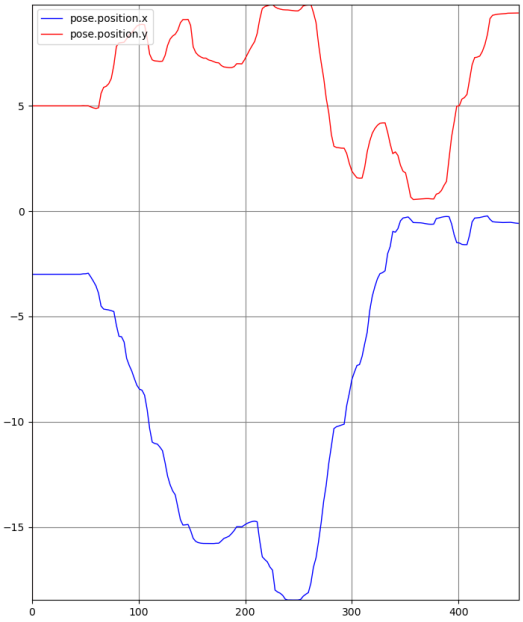
After doing so, we have redefined function send_goal utilizing the previously mentioned sender function and changing the order of the goal as follow:

Goal 3 → Goal 4 → Goal 2 → Goal 5 → Goal 1 → Goal 6 → Goal 7

```
. . .
if ( cmd == 1) {
            MoveBaseClient ac("move base", true);
            while(!ac.waitForServer(ros::Duration(5.0))){
            ROS_INFO("Waiting for the move_base action server to come up");
            }
            // 3 4 2 5 1 6 7

            sender( goal pos3, goal or3, 3, goal);
            ac.sendGoal(goal);
            ac.waitForResult();

            if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED){
                ROS_INFO("The mobile robot arrived in the TF goal 3");
                sender(_goal_pos4,_goal_or4, 4, goal);
                ac.sendGoal(goal);
                ac.waitForResult();

                if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED){
                    ROS_INFO("The mobile robot arrived in the TF goal 4");
                    sender( goal pos2, goal or2, 2, goal);
                    ac.sendGoal(goal);
                    ac.waitForResult();
. . .
```



We also record a bag fil of this trajectory and a video that you can find on the GitHub repo



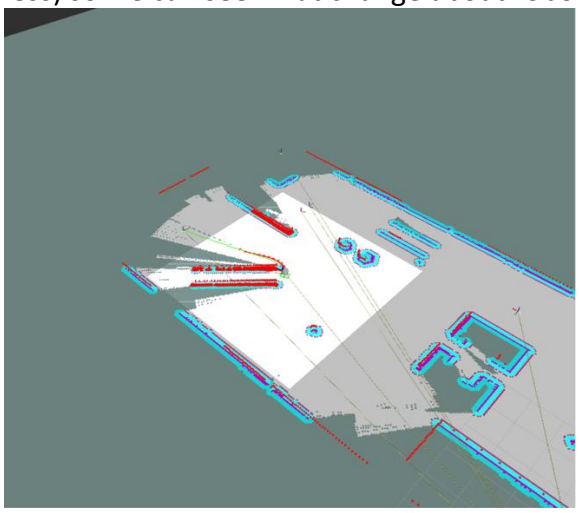
b) Change the parameters of the planner and move_base (try at least 4 different configurations) and comment on the results you get in terms of robot trajectories. The parameters that need to be changed are:

- In file teb_locl_planner_params.yaml: tune parameters related to the section about trajectory, robot, and obstacles.
- In file local_costmap_params.yaml and global_costmap_params.yaml: change dimensions' values and update costmaps' frequency.

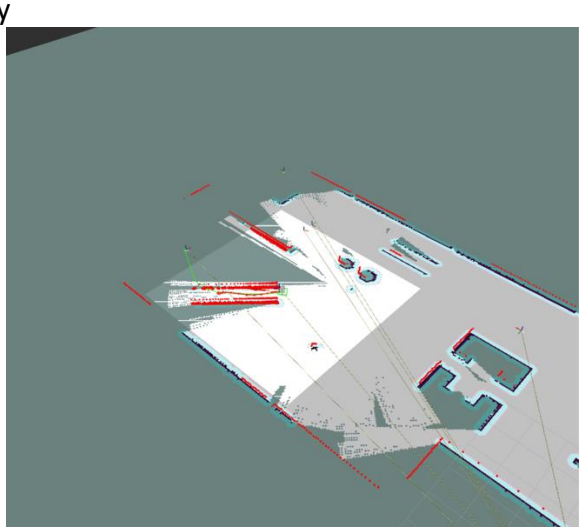
- In file costmap_common_params.yaml: tune parameters related to the obstacle and raytrace ranges and footprint coherently as done in planner parameters.

All tests for the configurations we will see below were performed by launching the tf_nav file.

Configuration n.1
For this configuration we consider the movement between the Goal 3 and the Goal 4. Inside the *teb_locl_planner_params.yaml* we reduce the value of min_obstacle_dist from 0.10 to 0.05 (or less) so we can see what change about robot trajectory



Base configuration

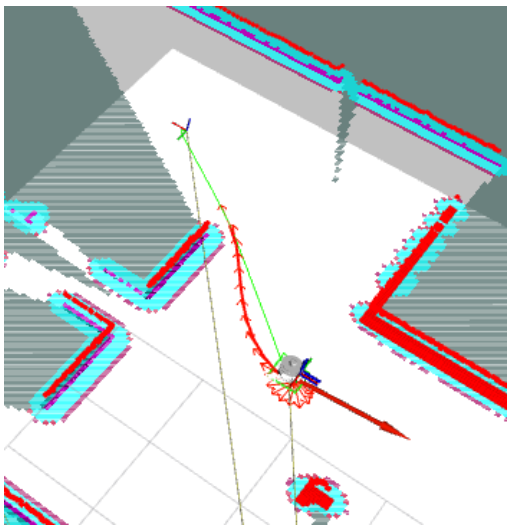


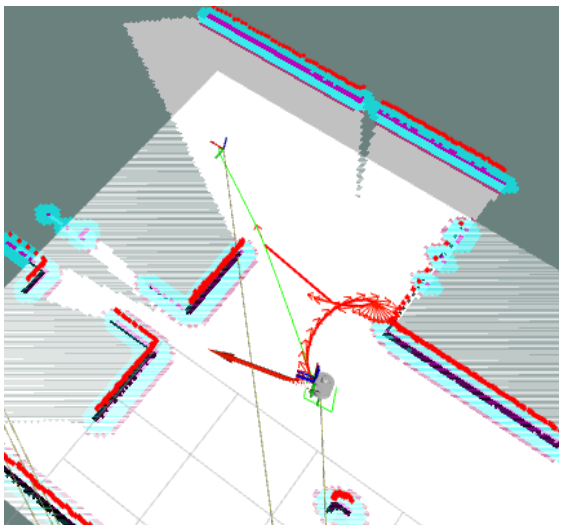
Configuration n1

As we can see by reducing the minimum value of the distance with the obstacles, our robot is able to pass through the tunnel1, which it could not do with the previous configuration since it decided to go around the obstacle.

Configuration n.2

In this configuration we change the value of min_obstacle_dist from 0.10 to 1 so we can see what change about robot trajectory; in the goal3, we see that the robot doesn't find a correct trajectory.





Configuration n.3

For this configuration we consider the movement between the Goal 0 and the Goal 3. Inside the *teb_locl_planner_params.yaml* we increment the value of max_vel_x from 0.6 to 1.2 and the value of acc_lim_x from 0.1 to 0.4 after that we also add a simple timer inside the tf_nav.cpp as follow

```
        if ( cmd == 1) {
            MoveBaseClient ac("move base", true);
            while(!ac.waitForServer(ros::Duration(5.0))){
            ROS_INFO("Waiting for the move base action server to come up");
            }

            sender( goal_pos3, goal_or3, 3, goal);
            ac.sendGoal(goal);
            ros::Time inizio_timer=ros::Time::now();
            ac.waitForResult();

            if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED){
                ROS_INFO("The mobile robot arrived in the TF goal 3");
                ros::Time fine_timer=ros::Time::now();
                cout<< "\n Timer: " << fine_timer-inizio_timer<< endl<<endl;
```

We carried out the tests and below we can see the results obtained.

```
[ INFO] [1702738358.079960881, 61.548000000]: The mobile robot arrived in the TF goal 3

 Timer: 32.420000000
```

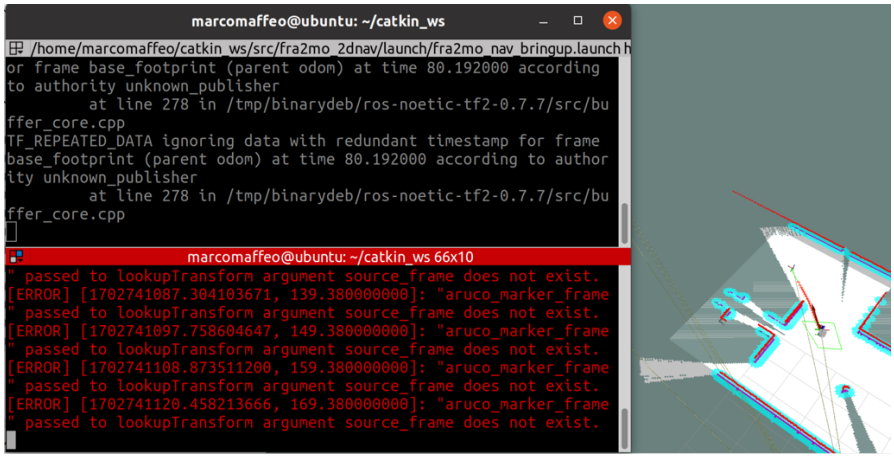
Base Configuration

```
[ INFO] [1702738145.307072312, 71.592000000]: The mobile robot arrived in the TF goal 3

 Timer: 27.208000000
```

Configuration n.2

Configuration n.4

For this configuration we consider the movement starting from the Goal 0. Inside the *costmap_common_params.yaml* we change the value inside the matrix of footprint in particular each value of 0.15 became 0.5 in order to increase the area occupied by the robot.



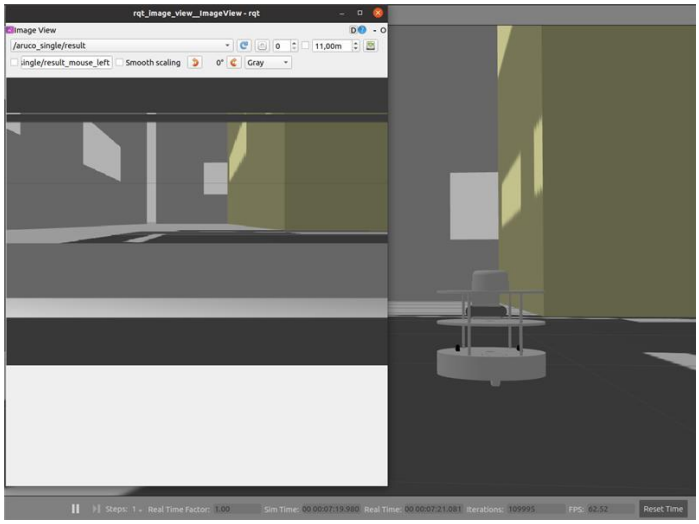
As we can see the area occupied by the robot is represented by the box in green and as a result we can see the robot is unable to reach the next goal since it gets stuck by the obstacle encountered by the newly increased footprint.

4. Vision-based navigation of the mobile platform

a) Run ArUco ROS node using the robot camera: bring up the camera model and uncomment it in that fra2mo.xacro file of the mobile robot description rl_fra2mo_description. Remember to install the camera description pkg: sudo apt-get install ros-<DISTRO>-realsense2-description

We have installed the camera description package using the command sudo apt-get install ros-noetic-realsense2-description. After that we uncomment the xacro plugin in the fra2mo.xacro file and send the following command on the terminator.

>> roslaunch aruco_ros usb_cam_aruco.launch camera:=/depth_camera/depth_camera



b) Implement a 2D navigation task following this logic

• Send the robot in the proximity of obstacle 9.

To send the robot in the proximity of obstacle 9 we decide to insert another command in the send_goal function that sends the robot directly to that goal

```
else if (cmd == 3){
            MoveBaseClient ac("move_base", true);
            while(!ac.waitForServer(ros::Duration(5.0))){
            ROS_INFO("Waiting for the move_base action server to come up");
            }
            goal.target_pose.header.frame_id = "map";
            goal.target_pose.header.stamp = ros::Time::now();
            goal.target_pose.pose.position.x = -15.5;
            goal.target_pose.pose.position.y = 7.5;
            goal.target_pose.pose.orientation.w = 0.0;
            goal.target_pose.pose.orientation.z = 1.0;
            ROS_INFO("Sending OBSTACLE 9 goal");
```

```
        ac.sendGoal(goal);
        ac.waitForResult();
        if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED){
           ROS_INFO("The mobile robot arrived in the OBSTACLE 9 position");
        }
        else
            ROS_INFO("The base failed to move to OBSTACLE 9 for some reason");
```

- Make the robot look for the ArUco marker. Once detected, retrieve its pose with respect to the map frame.

First of all, we changed in the sdf file of the marker 115 in rl_racefield the sizes of the tag, support and collision parameters:

```
<model name="marker_115">
  <static>true</static>
  <link name="marker">
    <gravity>false</gravity>
    <visual name="tag">
      <geometry>
        <box>
          <size>0.10 0.10 0.002</size>
        </box>
      </geometry>
      <material>
        <script>
          <uri>model://marker_115/material/scripts</uri>
          <uri>model://marker 115/material/textures</uri>
          <name>marker 115</name>
        </script>
      </material>
    </visual>
    <visual name="support">
      <geometry>
        <box>
          <size>0.12 0.12 0.001</size>
        </box>
      </geometry>
      <material>Gazebo/White</material>
    </visual>
    <collision name="collision">
      <geometry>
        <box>
          <size>0.12 0.12 0.001</size>
        </box>
      </geometry>
    </collision>
```

In the usb_cam_aruco.launch we change markerId, markerSize, camera, ref_frame and camera_frame arguments as follow:

```
<launch>

    <arg name="markerId"        default="115"/>
    <arg name="markerSize"      default="0.1"/>    <!-- in m -->
    <arg name="camera"          default="depth_camera/depth_camera"/>
    <arg name="marker_frame"    default="aruco_marker_frame"/>
    <arg name="ref_frame"       default="map"/>
    <arg name="corner_refinement" default="LINES" />

    <node pkg="aruco_ros" type="single" name="aruco_single">
        <remap from="/camera_info" to="/$(arg camera)/camera_info" />
        <remap from="/image" to="/$(arg camera)/image_raw" />
        <param name="image_is_rectified" value="True"/>
        <param name="marker_size"        value="$(arg markerSize)"/>
        <param name="marker_id"          value="$(arg markerId)"/>
        <param name="reference_frame"    value="$(arg ref_frame)"/>
        <param name="camera_frame"       value="camera_depth_optical_frame"/>
        <param name="marker_frame"       value="$(arg marker_frame)" />
        <param name="corner_refinement"  value="$(arg corner_refinement)" />
    </node>

</launch>
```

In this way during the navigation of the robot the frame of the aruco marker doesn't move and we retrieve its pose with respect to the map frame.

To save the aruco marker position and orientation we define two global variables inside the tf_nav.cpp:

```
Eigen::Vector3d aruco_pose;
Eigen::Vector4d aruco_or;
```

We define a new thread inside the previous code that sand the robot near the obstacle 9, calling the function marker_listner wrote using as example the previous homework in which we retrive the pose of the aruco marker

```
else if (cmd == 3){
            MoveBaseClient ac("move_base", true);
            while(!ac.waitForServer(ros::Duration(5.0))){
            ROS_INFO("Waiting for the move_base action server to come up");
            }
            boost::thread tf_listener_aruco_t( &TF_NAV::marker_listener, this );

            goal.target_pose.header.frame_id = "map";
            goal.target_pose.header.stamp = ros::Time::now();
            goal.target_pose.pose.position.x = -15.5;
```

```
            goal.target_pose.pose.position.y = 7.5;
            goal.target_pose.pose.orientation.w = 0.0;
            goal.target_pose.pose.orientation.z = 1.0;

            ROS_INFO("Sending OBSTACLE 9 goal");
            ac.sendGoal(goal);
```

```
void TF_NAV::marker_listener(){
    ros::Rate r( 1 );
    tf::TransformListener listener;
    tf::StampedTransform transform;
    while ( ros::ok() )
    {
        try
        {
    listener.waitForTransform( "map", "aruco_marker_frame", ros::Time( 0 ), ros::Duration( 10.0 ) );
    listener.lookupTransform( "map", "aruco_marker_frame", ros::Time( 0 ), transform );
        }
        catch( tf::TransformException &ex )
        {
            ROS_ERROR("%s", ex.what());
            r.sleep();
            continue;
        }
       aruco_pose << transform.getOrigin().x(), transform.getOrigin().y(),
transform.getOrigin().z();
        aruco_or << transform.getRotation().w(),  transform.getRotation().x(),
transform.getRotation().y(), transform.getRotation().z();
        r.sleep();
    }
```

- Set the following pose (relative to the ArUco marker pose) as next goal for the robot

$$x = x_m + 1, \quad y = y_m,$$

where x_m, y_m are the marker coordinates.

To send the robot to the next pose, after the robot successfully arrived in the proximity of obstacle 9, we set as new goal the desired pose

```
if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED){
       ROS_INFO("The mobile robot arrived in the OBSTACLE 9 position");

       goal.target pose.header.frame id = "map";
       goal.target_pose.header.stamp = ros::Time::now();
       goal.target pose.pose.position.x = aruco pose[0] + 2.0;
       goal.target_pose.pose.position.y = aruco_pose[1];
       ac.sendGoal(goal);
```

```
        ac.waitForResult();

        if(ac.getState() == actionlib::SimpleClientGoalState::SUCCEEDED){
            ROS_INFO("The mobile robot arrived in the MARKER position");
            }
        else
            ROS_INFO("The base failed to move to MARKER for some reason");
```

c) Publish the ArUco pose as TF following the example at this link.

To publish the ArUco pose as TF we declare a new function that take as input the position and the orientation of the aruco marker and broadcast its transform:

```
void poseCallback(Eigen::Vector3d aruco_pose, Eigen::Vector4d aruco_or){

   static tf::TransformBroadcaster br;
   tf::Transform transform;
   transform.setOrigin( tf::Vector3(aruco_pose[0], aruco_pose[1], aruco_pose[2]));
   tf::Quaternion q(aruco_or[1], aruco_or[2], aruco_or[3], aruco_or[0]);
   transform.setRotation(q);
   br.sendTransform(tf::StampedTransform(transform, ros::Time::now(), "map", "aruco_pose_tf"));

}
```

We call this function inside the send_goal function

```
while ( ros::ok() )
    {
        poseCallback(aruco_pose,aruco_or);  //tf publisher aurco pose

        std::cout<<"\nInsert 1 to send goal from TF "<<std::endl;
        std::cout<<"Insert 2 to send home position goal "<<std::endl;
        std::cout<<"Insert 3 to send marker position goal "<<std::endl;
        std::cout<<"Inser your choice"<<std::endl;
        std::cin>>cmd;
```

We record a video where the robot goes neary the obstacle 9, look at the aruco marker and retrive the aruco marker pose; after that the robot goes at the final position with x = xm + 2.0 because the marker is too high and the robot cannot see it.
Once it arrived to the final goal, the aruco_pose_tf has been published as static TF.
You can see that video on the GitHub repository.